```
free ch_sensor_init : channel [private].
free ch_user_reg    : channel [private].
free ch_fog_reg     : channel [private].
free KF   : bitstring [private].
free KS   : bitstring [private].
free PWU  : bitstring [private].
free BIOU : bitstring [private].
free IDU : bitstring.
free IDF : bitstring.
free IDS : bitstring.
fun h(bitstring)                    : bitstring.
fun hmac(bitstring, bitstring)      : bitstring.
fun xor(bitstring, bitstring)       : bitstring.
equation forall a : bitstring, b : bitstring;
  xor(xor(a, b), b) = a.
fun pair(bitstring, bitstring) : bitstring.
fun fst(bitstring)             : bitstring.
fun snd(bitstring)             : bitstring.
equation forall a : bitstring, b : bitstring; fst(pair(a,b)) = a.
equation forall a : bitstring, b : bitstring; snd(pair(a,b)) = b.
event UserRegistered(bitstring).
event FogRegisteredUser(bitstring).
event UserInitiatesAuth(bitstring, bitstring).
event FogVerifiesUser(bitstring, bitstring).
event FogForwardsToSensor(bitstring, bitstring).
event SensorVerifiesFog(bitstring, bitstring).
event SensorEstablishesSK(bitstring).
event FogEstablishesSK(bitstring).
event UserEstablishesSK(bitstring).
query attacker(KF).
query attacker(KS).
query attacker(PWU).
query attacker(BIOU).
free sk_secret : bitstring [private].
query attacker(sk_secret).
query pidu : bitstring, m2 : bitstring;
  inj-event(FogVerifiesUser(pidu, m2))
    ==> inj-event(UserInitiatesAuth(pidu, m2)).
```

```
query m4 : bitstring, nu2 : bitstring;
  inj-event(SensorVerifiesFog(m4, nu2))
    ==> inj-event(FogForwardsToSensor(m4, nu2)).
query sk : bitstring;
  event(FogEstablishesSK(sk)) ==> event(SensorEstablishesSK(sk)).
query sk : bitstring;
  event(UserEstablishesSK(sk)) ==> event(FogEstablishesSK(sk)).
let FogSensorInit() =
  let M1s = h((IDS, KS)) in
  let M2s = h((M1s, KF)) in
  out(ch_sensor_init, (IDS, M1s, M2s, KF)).
let SensorReceiveCreds() =
  in(ch_sensor_init,
     (ids_in : bitstring, m1s_in : bitstring,
      m2s_in : bitstring, kf_in  : bitstring));
  out(ch_sensor_init, (ids_in, m1s_in, m2s_in, kf_in)).
let UserRegistration() =
  new nU1 : bitstring;
  let AuthU = h((IDU, PWU, nU1))  in
  let BioU  = h((BIOU, nU1))      in
  out(sec, (IDU, AuthU, BioU));
  in(sec, (PIDU : bitstring, Reg1 : bitstring,
           Reg2 : bitstring, Reg3 : bitstring));
  event UserRegistered(PIDU);
  out(ch_user_reg, (PIDU, Reg1, Reg2, Reg3, nU1)).
let FogRegistration() =
  in(sec, (idu : bitstring, authU_r : bitstring, bioU_r : bitstring));
  new PIDU : bitstring;
  new nF1  : bitstring;
  let hPK  = h((PIDU, KF))             in
  let Reg1 = xor(hPK, authU_r)         in
  let Reg2 = xor(hPK, nF1)             in
  let Reg3 = h((authU_r, nF1, bioU_r)) in
  out(sec, (PIDU, Reg1, Reg2, Reg3));
  event FogRegisteredUser(PIDU);
  out(ch_fog_reg, (PIDU, nF1, authU_r, bioU_r)).
let UserAuth() =
  (* --- Step 1 --- *)
  in(ch_user_reg,
     (PIDU : bitstring, Reg1 : bitstring,
      Reg2 : bitstring, Reg3 : bitstring, nU1 : bitstring));
  let AuthU = h((IDU, PWU, nU1))    in
```

```
  let BioU  = h((BIOU, nU1))        in
  let hPK   = xor(Reg1, AuthU)      in
  let nF1   = xor(Reg2, hPK)        in
  let Reg3p = h((AuthU, nF1, BioU)) in
  if Reg3p = Reg3 then
  new nU2 : bitstring;
  new TS1 : bitstring;
  let authbio = pair(AuthU, BioU)                    in
  let M1_u    = xor(nU2, hPK)                        in
  let M2_u    = hmac(authbio, (PIDU, IDS, TS1))      in
  event UserInitiatesAuth(PIDU, M2_u);
  out(pub, (PIDU, M1_u, M2_u, TS1));
  (* --- Step 5 --- *)
  in(pub, (Reg1n : bitstring, Reg2n  : bitstring,
           PIDUn : bitstring, M8     : bitstring,
           TS7   : bitstring, enc_n  : bitstring));
  let nonces_p = xor(enc_n, hPK)       in
  let nF2_u    = fst(nonces_p)         in
  let nS1_u    = snd(nonces_p)         in
  let SK_u = h((nU2, nF2_u, nS1_u, AuthU, BioU)) in
  let M8p  = hmac(SK_u, (Reg1n, Reg2n, PIDUn, TS7)) in
  if M8p = M8 then
  event UserEstablishesSK(SK_u).
let FogAuth() =
  in(ch_fog_reg,
     (PIDU_db  : bitstring, nF1_db   : bitstring,
      authU_db : bitstring, bioU_db  : bitstring));
  (* --- Step 2 --- *)
  in(pub, (PIDU_r : bitstring, M1_r  : bitstring,
           M2_r   : bitstring, TS1_r : bitstring));
  if PIDU_r = PIDU_db then
  let hPK_f    = h((PIDU_r, KF))              in
  let nU2_r    = xor(M1_r, hPK_f)             in
  let authU_f  = authU_db                     in
  let bioU_f   = bioU_db                      in
  let authbio_f = pair(authU_f, bioU_f)       in
  let M2p = hmac(authbio_f, (PIDU_r, IDS, TS1_r)) in
  if M2p = M2_r then
  event FogVerifiesUser(PIDU_r, M2_r);
  let M1s_f  = h((IDS, KS))   in
  let M2s_f  = h((M1s_f, KF)) in
  new nF2 : bitstring;
```

```
  new TS3 : bitstring;
  let M3_f = xor(nF2, M2s_f)                      in
  let M4_f = hmac(M1s_f, (IDF, IDU, nU2_r, TS3))  in
  let M5_f = xor(authbio_f, h((nF2, M1s_f)))       in
  event FogForwardsToSensor(M4_f, nU2_r);
  out(pub, (M3_f, M4_f, M5_f, TS3, nU2_r));
  (* --- Step 4 --- *)
  in(pub, (M6_r : bitstring, M7_r : bitstring, TS5_r : bitstring));
  let nS1_r  = xor(M6_r, h((nU2_r, nF2)))            in
  let SK_f   = h((nU2_r, nF2, nS1_r, authU_f, bioU_f)) in
  let M7p    = hmac(SK_f, (IDS, IDF, TS5_r))          in
  if M7p = M7_r then
  event FogEstablishesSK(SK_f);
  new PIDUn : bitstring;
  new TS7   : bitstring;
  let hPKn  = h((PIDUn, KF))                          in
  let Reg1n = xor(hPKn, authbio_f)                    in
  let Reg2n = xor(hPKn, nF1_db)                       in
  let M8_f  = hmac(SK_f, (Reg1n, Reg2n, PIDUn, TS7))  in
  let enc_n = xor(pair(nF2, nS1_r), hPK_f)            in
  out(pub, (Reg1n, Reg2n, PIDUn, M8_f, TS7, enc_n)).
let SensorAuth() =
  in(ch_sensor_init,
     (ids_s : bitstring, M1s_s : bitstring,
      M2s_s : bitstring, kf_s  : bitstring));
  in(pub, (M3_r  : bitstring, M4_r  : bitstring,
           M5_r  : bitstring, TS3_r : bitstring,
           nU2_r : bitstring));
  let M4p = hmac(M1s_s, (IDF, IDU, nU2_r, TS3_r)) in
  if M4p = M4_r then
  event SensorVerifiesFog(M4_r, nU2_r);
  let nF2_s     = xor(M3_r, M2s_s)                in
  let authbio_s = xor(M5_r, h((nF2_s, M1s_s)))   in
  new nS1 : bitstring;
  new TS5 : bitstring;
  let SK_s = h((nU2_r, nF2_s, nS1, authbio_s))   in
  event SensorEstablishesSK(SK_s);
  let M6_s = xor(nS1, h((nU2_r, nF2_s)))         in
  let M7_s = hmac(SK_s, (IDS, IDF, TS5))          in
  out(pub, (M6_s, M7_s, TS5)).
process
```

```
  FogSensorInit()      | SensorReceiveCreds() |  UserRegistration()   |  FogRegistration()    |  UserAuth()
            |  FogAuth()            |  SensorAuth()
```

upon running the code using http://proverif24.paris.inria.fr/, copy the code, paste in the said url, the following result will be displayed.

```
--------------------------------------------------------------
Verification summary:

Query not attacker(KF[]) is true.

Query not attacker(KS[]) is true.

Query not attacker(PWU[]) is true.

Query not attacker(BIOU[]) is true.

Query not attacker(sk_secret[]) is true.

Query inj-event(FogVerifiesUser(pidu,m2)) ==> inj-event(UserInitiatesAuth(pidu,m2)) is true.

Query inj-event(SensorVerifiesFog(m4,nu2)) ==> inj-event(FogForwardsToSensor(m4,nu2)) is true.

Query event(FogEstablishesSK(sk)) ==> event(SensorEstablishesSK(sk)) is true.

Query event(UserEstablishesSK(sk)) ==> event(FogEstablishesSK(sk)) is true.

--------------------------------------------------------------
```